

PRÁCTICA DE LABORATORIO

De los Datos Crudos al Contador de Pasos

Sensores inerciales MEMS · Calibración · Filtrado · PDR

APS-GRIA · Universidad de Vigo

Curso 2025–2026 · Duración: 2 horas

Hardware: M5StickC Plus 2 · Firmware: UIFlow 2.0 / MicroPython

Bloque	Título	Duración	Objetivo principal
1	Observación y ruido	20 min	Entender acelerómetro, giroscopio y bias
2	Calibración por software	25 min	Eliminar el error de offset del giroscopio
3	Magnitud vectorial	25 min	Señal invariante a orientación
4	Detector de pasos	50 min	Algoritmo de pico completo y ajuste empírico

Introducción

En esta práctica partimos del código `IMU.py`, que muestra en tiempo real los datos del acelerómetro (`Acc_x/y/z`) y del giroscopio (`Gyr_x/y/z`) del chip MPU6886 integrado en el M5StickC Plus 2. El objetivo es recorrer de forma progresiva el camino que va desde los datos crudos hasta un algoritmo de detección de pasos funcional, aplicando los conceptos del Tema 3 (física MEMS, ruido, bias, deriva y PDR).

La práctica se divide en **4 bloques independientes pero encadenados**: cada bloque añade una capa de comprensión y código sobre el anterior. Al final compararéis vuestro detector con el fichero `contador_pasos.py` del proyecto, que implementa filtros de banda más sofisticados.

i Material necesario

- M5StickC Plus 2 con firmware UIFlow 2.0 (MicroPython)
- PC con UIFlow Web IDE o Thonny conectado por USB/WiFi
- Ficheros del proyecto: `IMU.py`, `contador_pasos.py` (referencia final)
- Manual UIFlow 2.0 — Sección 6 (IMU) y Sección 13 (time)
- TEMA3_ANEXO.pdf — Marco teórico de referencia

**BLOQUE
1****Observación y comprensión del ruido** **20 min**

1.1 Objetivo

Antes de escribir ningún algoritmo, es fundamental **observar y anotar** qué produce el sensor en distintas condiciones físicas. Este bloque no requiere modificar el código: trabajáis únicamente con el `IMU.py` original.

1.2 Código de partida (sin modificar)

El fichero `IMU.py` ya está disponible en el proyecto. Cargadlo en el M5StickC y observad la pantalla en tiempo real:

```
PYTHON
# IMU.py - código de partida (sin cambios)
import os, sys, io
import M5
from M5 import *

label_ax = label_ay = label_az = None
label_gx = label_gy = label_gz = None

def setup():
    global label_ax, label_ay, label_az, label_gx, label_gy, label_gz
    M5.begin()
    Widgets.fillScreen(0x222222)
    Widgets.Title('IMU Demo', 3, 0xFFFFFF, 0x0000FF, Widgets.FONTS.DejaVu18)
    label_ax = Widgets.Label('Acc_x:', 1, 30, 1.0, 0xFFFFFF, 0x222222, Widgets.FONTS.DejaVu12)
    label_ay = Widgets.Label('Acc_y:', 1, 50, 1.0, 0xFFFFFF, 0x222222, Widgets.FONTS.DejaVu12)
    label_az = Widgets.Label('Acc_z:', 1, 70, 1.0, 0xFFFFFF, 0x222222, Widgets.FONTS.DejaVu12)
    label_gx = Widgets.Label('Gyr_x:', 1, 95, 1.0, 0x00FFFF, 0x222222, Widgets.FONTS.DejaVu12)
    label_gy = Widgets.Label('Gyr_y:', 1, 115, 1.0, 0x00FFFF, 0x222222, Widgets.FONTS.DejaVu12)
    label_gz = Widgets.Label('Gyr_z:', 1, 135, 1.0, 0x00FFFF, 0x222222, Widgets.FONTS.DejaVu12)

def loop():
    M5.update()
    acc = Imu.getAccel()
    gyro = Imu.getGyro()
    label_ax.setText('Acc_x: {:.2f}'.format(acc[0]))
    label_ay.setText('Acc_y: {:.2f}'.format(acc[1]))
    label_az.setText('Acc_z: {:.2f}'.format(acc[2]))
    label_gx.setText('Gyr_x: {:.2f}'.format(gyro[0]))
    label_gy.setText('Gyr_y: {:.2f}'.format(gyro[1]))
    label_gz.setText('Gyr_z: {:.2f}'.format(gyro[2]))
```

1.3 Experimentos de observación

Realizad los siguientes experimentos y rellenad la tabla de resultados. Anotad los valores observados en pantalla:

Experimento A — M5 plano sobre la mesa

Colocad el M5StickC completamente plano sobre la mesa, pantalla hacia arriba. Waited 3 segundos hasta que los valores se estabilicen y anotad.

Variable	Valor esperado (teoría)	Valor medido	Explicación
Acc_x	≈ 0 g	_____	Sin aceleración horizontal
Acc_y	≈ 0 g	_____	Sin aceleración horizontal
Acc_z	≈ +1 g	_____	Gravedad apunta hacia Z
Gyr_x	≈ 0 °/s	_____	Sin rotación
Gyr_y	≈ 0 °/s	_____	Sin rotación

Gyr_z	≈ 0 °/s	_____	Sin rotación
-------	-----------------	-------	--------------

i Concepto clave: Fuerza específica

El acelerómetro NO mide aceleración del movimiento puro; mide la fuerza de reacción que recibe.

En reposo sobre una mesa, la mesa empuja el chip hacia arriba con 1g para contrarrestar la gravedad.

Por eso $\text{Acc}_z \approx +1$ g aunque el dispositivo esté completamente quieto.

En caída libre (cero fuerzas externas), el acelerómetro marcaría 0 g en todos los ejes.

Experimento B — Rotación de 90° sobre el eje X

Girad el M5 90° hacia adelante (como si cayera hacia la pantalla). La gravedad ahora apunta en dirección al eje Y del chip. Anotad:

Variable	Valor aprox. antes	Valor aprox. después	¿Qué cambió?
Acc_x	≈ 0	_____	
Acc_y	≈ 0	_____	
Acc_z	≈ 1 g	_____	

Experimento C — Cálculo manual de magnitud

Con los valores del Experimento A, calculad a mano la **magnitud vectorial**:

$$|a| = \sqrt{(\text{Acc}_x^2 + \text{Acc}_y^2 + \text{Acc}_z^2)} = \sqrt{(\quad^2 + \quad^2 + \quad^2)} =$$

Ahora repetid el cálculo con los valores del Experimento B (girado 90°):

$$|a| = \sqrt{(\text{Acc}_x^2 + \text{Acc}_y^2 + \text{Acc}_z^2)} = \sqrt{(\quad^2 + \quad^2 + \quad^2)} =$$

☒ Pregunta de reflexión 1

¿Son iguales las dos magnitudes aunque el dispositivo esté orientado de forma diferente?

¿Por qué esto es útil para detectar pasos independientemente de cómo lleva el usuario el M5?

Respuesta esperada: la magnitud es ≈ 1 g en ambos casos porque es invariante a la rotación.

Experimento D — Identificación del bias del giroscopio

Dejad el M5 completamente quieto sobre la mesa durante 10 segundos. Observad los valores del giroscopio y anotad el eje con mayor desviación de cero:

Eje giroscopio	Valor en reposo observado	¿Es cero exacto?	Nombre del error
Gyr_x	_____		Bias / Offset
Gyr_y	_____		Bias / Offset
Gyr_z	_____		Bias / Offset

¿Por qué existe el bias?

El silicio de los muelles microscópicos tiene imperfecciones de fabricación: pequeñas asimetrías

en los peines capacitivos hacen que el circuito ASIC 'vea' un desequilibrio aunque no haya movimiento.

Este error es constante en el corto plazo pero deriva lentamente con la temperatura (drift térmico).

El Tema 3 muestra que si integramos un bias de 0.01g durante 30s, el error de posición es de 45 metros.

1.4 Resumen del bloque

- El eje Z del acelerómetro detecta $\sim 1g$ en reposo porque mide la reacción normal de la superficie.
- La magnitud vectorial $\sqrt{x^2+y^2+z^2} \approx 1g$ con independencia de la orientación del dispositivo.
- El giroscopio tiene un bias (offset) que produce un valor distinto de cero aunque no haya rotación.
- Este bias, si se integra para obtener ángulo o posición, genera errores que crecen cuadráticamente.

**BLOQUE
2****Calibración estática del giroscopio por software** **25 min**

2.1 Objetivo

Implementar una **rutina de calibración estática** que mida el error en reposo al inicio y lo reste en todas las lecturas posteriores. Al final del bloque, los valores del giroscopio deben ser ≈ 0 °/s con el dispositivo quieto.

2.2 Fundamento teórico

El modelo matemático del sensor real (Tema 3) es:

$$x_{\text{medido}}[n] = S \cdot x_{\text{real}}[n] + B[n] + N[n]$$

Donde:

- S = sensibilidad (ganancia, idealmente = 1)
- $B[n]$ = bias (offset). En el corto plazo es constante; a largo plazo deriva con la temperatura
- $N[n]$ = ruido térmico de media cero (Ruido Blanco de Johnson-Nyquist)

La **calibración estática** consiste en medir B estimando $B \approx$ promedio de muchas muestras en reposo, ya que el ruido $N[n]$ tiene media cero y se cancela al promediar.

$$B_{\text{estimado}} = (1/N) \cdot \sum x_{\text{medido}}[n] \quad (\text{con el dispositivo quieto})$$

¿Cuántas muestras necesitamos?

Con 100 muestras a 50 Hz (2 segundos de espera) obtenemos una estimación de B con error < 0.1 °/s.

Más muestras = más precisión, pero más tiempo de espera al inicio.

200 muestras (4 segundos) es un buen compromiso para MPU6886 de bajo coste.

2.3 Código completo del bloque 2

Cread un nuevo fichero llamado `imu_calibrado.py` con el siguiente contenido:

```
PYTHON
import os, sys, io
import M5
from M5 import *
import time

# --- Widgets globales ---
title0 = label_ax = label_ay = label_az = None
label_gx = label_gy = label_gz = None
label_status = None

# --- Variables de calibración ---
# Almacenan el promedio del error medido en reposo
bias_gx = 0.0
bias_gy = 0.0
bias_gz = 0.0
calibrado = False

# --- Rutina de calibración ---
def calibrar_giroscopio(n_muestras=200):
    """
    Calcula el bias del giroscopio promediando n_muestras
    con el dispositivo completamente quieto.
    Devuelve (bias_x, bias_y, bias_z) en grados/segundo.
    """
    global label_status
    label_status.setText('Calibrando... NO MOVER')
    label_status.setColor(0xFF8800, 0x222222)

    suma_x = 0.0
```

```

suma_y = 0.0
suma_z = 0.0

for i in range(n_muestras):
    g = Imu.getGyro()          # (gx, gy, gz) en dps
    suma_x += g[0]
    suma_y += g[1]
    suma_z += g[2]

    # Mostrar progreso cada 20 muestras
    if i % 20 == 0:
        pct = int(i / n_muestras * 100)
        label_status.setText('Calibrando {}%'.format(pct))

    time.sleep_ms(10)          # 100 Hz durante la calibración

b_x = suma_x / n_muestras
b_y = suma_y / n_muestras
b_z = suma_z / n_muestras

label_status.setText('Calibrado OK!')
label_status.setColor(0x00FF00, 0x222222)
return b_x, b_y, b_z

# — Setup —————
def setup():
    global title0, label_ax, label_ay, label_az
    global label_gx, label_gy, label_gz, label_status
    global bias_gx, bias_gy, bias_gz, calibrado

    M5.begin()
    Widgets.fillScreen(0x222222)

    title0 = Widgets.Title('IMU Calibrado', 3, 0xFFFFFF, 0x0000FF, Widgets.FONTS.DejaVu18)
    label_ax = Widgets.Label('Acc_x:', 1, 30, 1.0, 0xFFFFFF, 0x222222, Widgets.FONTS.DejaVu12)
    label_ay = Widgets.Label('Acc_y:', 1, 50, 1.0, 0xFFFFFF, 0x222222, Widgets.FONTS.DejaVu12)
    label_az = Widgets.Label('Acc_z:', 1, 70, 1.0, 0xFFFFFF, 0x222222, Widgets.FONTS.DejaVu12)
    label_gx = Widgets.Label('Gyr_x:', 1, 95, 1.0, 0x00FFFF, 0x222222, Widgets.FONTS.DejaVu12)
    label_gy = Widgets.Label('Gyr_y:', 1, 115, 1.0, 0x00FFFF, 0x222222, Widgets.FONTS.DejaVu12)
    label_gz = Widgets.Label('Gyr_z:', 1, 135, 1.0, 0x00FFFF, 0x222222, Widgets.FONTS.DejaVu12)
    label_status = Widgets.Label('', 1, 158, 1.0, 0xFF8800, 0x222222, Widgets.FONTS.DejaVu12)

    # Ejecutar calibración al inicio (dispositivo en reposo)
    bias_gx, bias_gy, bias_gz = calibrar_giroscopio(n_muestras=200)
    calibrado = True

    # Mostrar los valores de bias encontrados en consola
    print('Bias encontrado -> Gx:{:.3f} Gy:{:.3f} Gz:{:.3f}'.format(
        bias_gx, bias_gy, bias_gz))

# — Loop —————
def loop():
    global label_ax, label_ay, label_az
    global label_gx, label_gy, label_gz
    global bias_gx, bias_gy, bias_gz

    M5.update()

    acc = Imu.getAccel()
    gyro = Imu.getGyro()

    # Corrección de bias: restamos el error medido en calibración
    gx_cal = gyro[0] - bias_gx
    gy_cal = gyro[1] - bias_gy
    gz_cal = gyro[2] - bias_gz

    # Mostrar acelerómetro sin cambios
    label_ax.setText('Acc_x: {:.2f}'.format(acc[0]))
    label_ay.setText('Acc_y: {:.2f}'.format(acc[1]))
    label_az.setText('Acc_z: {:.2f}'.format(acc[2]))

    # Mostrar giroscopio YA CALIBRADO
    label_gx.setText('Gyr_x: {:.2f}'.format(gx_cal))
    label_gy.setText('Gyr_y: {:.2f}'.format(gy_cal))
    label_gz.setText('Gyr_z: {:.2f}'.format(gz_cal))

```

```

# Recalibrar si se mantiene pulsado BtnA
if BtnA.wasHold():
    bias_gx, bias_gy, bias_gz = calibrar_giroscopio(200)

# — Main —
if __name__ == '__main__':
    try:
        setup()
        while True:
            loop()
    except (Exception, KeyboardInterrupt) as e:
        try:
            from utility import print_error_msg
            print_error_msg(e)
        except ImportError:
            print('Error:', e)

```

2.4 Experimentos de verificación

Experimento A — Comparativa antes y después

Eje	Valor con IMU.py (sin calibrar)	Valor con imu_calibrado.py	Reducción
Gyr_x	_____	_____	_____
Gyr_y	_____	_____	_____
Gyr_z	_____	_____	_____

Experimento B — Prueba de movimiento real

Con el código calibrado, realizad un giro de 360° sobre el eje Z a velocidad constante (aproximadamente 1 segundo para la vuelta completa). El valor de Gyr_z debería marcar ≈ 360 °/s durante ese segundo.

☒ Pregunta de reflexión 2

¿Desaparecen completamente los valores del giroscopio en reposo o queda un pequeño residuo?

¿Qué ocurriría si el dispositivo se calienta 10 minutos y volvéis a mirarlo? (drift térmico)

¿Podría solucionarse recalibrando periódicamente? ¿Cuál es el inconveniente?

2.5 Funcionalidad adicional: recalibración

El código añade la posibilidad de **recalibrar manteniendo pulsado BtnA** (pulsación larga). Esto simula la detección de estado estático que se usa en aplicaciones PDR: cuando el usuario para unos segundos, el sistema recalibra el giroscopio automáticamente. Probadlo calentando el chip en la mano 2 minutos y observando la deriva.

**BLOQUE
3****Magnitud vectorial e indicador visual de actividad** 25 min

3.1 Objetivo

Añadir al código de calibración el cálculo de la **magnitud vectorial** del acelerómetro y un indicador de actividad basado en umbrales. Esta es la señal "master" invariante a la orientación que usará el detector de pasos.

3.2 Fundamento: por qué usar la magnitud

La aceleración raw depende de cómo el usuario lleva el M5StickC. Si el usuario gira el dispositivo, la gravedad se redistribuye entre los tres ejes. Sin embargo, la magnitud euclidiana es invariante a cualquier rotación:

$$|a| = \sqrt{a_x^2 + a_y^2 + a_z^2} \rightarrow \text{siempre} \approx 1g \text{ en reposo, independientemente de la orientación}$$

Al caminar, el impacto del talón produce picos de 1.2–1.8g. Al restar la gravedad obtenemos la 'aceleración del usuario':

$$\text{user_accel} = |a| - 1.0g \rightarrow \approx 0 \text{ en reposo, } > 0 \text{ en movimiento}$$

3.3 Código completo del bloque 3

Guardad este fichero como imu_magnitud.py (incluye la calibración del bloque 2):

```
PYTHON
import os, sys, io
import M5
from M5 import *
import time
import math          # <-- NUEVO: para sqrt()

# --- Widgets ---
title0 = label_acc = label_mag = label_user = label_estado = label_status = None

# --- Calibración ---
bias_gx = bias_gy = bias_gz = 0.0

# --- Umbrales de actividad (ajustar empíricamente) ---
UMBRAL_QUIETO = 0.05 # user_accel < 0.05g -> quieto
UMBRAL_LEVE = 0.15 # user_accel < 0.15g -> movimiento leve
UMBRAL_CAMINA = 0.35 # user_accel < 0.35g -> caminando
# user_accel >= 0.35g -> movimiento intenso

def calibrar_giroscopio(n=200):
    label_status.setText('Calibrando...')
    label_status.setColor(0xFF8800, 0x222222)
    sx = sy = sz = 0.0
    for _ in range(n):
        g = Imu.getGyro()
        sx += g[0]; sy += g[1]; sz += g[2]
        time.sleep_ms(10)
    label_status.setText('OK')
    label_status.setColor(0x00FF00, 0x222222)
    return sx/n, sy/n, sz/n

def setup():
    global title0, label_acc, label_mag, label_user, label_estado, label_status
    global bias_gx, bias_gy, bias_gz

    M5.begin()
    Widgets.fillScreen(0x222222)

    title0 = Widgets.Title('IMU Magnitud', 3, 0xFFFFFF, 0x0000FF, Widgets.FONTS.DejaVu18)
    label_acc = Widgets.Label('Acc: --', 5, 35, 1.0, 0xFFFFFF, 0x222222,
    Widgets.FONTS.DejaVu12)
```

```
label_mag = Widgets.Label('|a|: --', 5, 58, 1.0, 0xFFFF00, 0x222222,
Widgets.FONTS.DejaVu18)
label_user = Widgets.Label('usr: --', 5, 88, 1.0, 0xFF8800, 0x222222,
Widgets.FONTS.DejaVu12)
label_estado= Widgets.Label('ESTADO', 5, 115, 1.0, 0x00FF00, 0x222222,
Widgets.FONTS.DejaVu18)
label_status= Widgets.Label('', 5, 145, 1.0, 0x888888, 0x222222,
Widgets.FONTS.DejaVu12)

bias_gx, bias_gy, bias_gz = calibrar_giroscopio(200)

def loop():
    M5.update()

    acc = Imu.getAccel()
    ax, ay, az = acc[0], acc[1], acc[2]

    # 1. Magnitud vectorial (invariante a orientación)
    mag = math.sqrt(ax*ax + ay*ay + az*az)

    # 2. Eliminar gravedad -> aceleración propia del usuario
    user_accel = abs(mag - 1.0)

    # 3. Clasificar estado por umbral
    if user_accel < UMBRAL_QUIETO:
        estado = 'QUIETO'
        color = 0x00FF00 # verde
    elif user_accel < UMBRAL_LEVE:
        estado = 'LEVE'
        color = 0xFFFF00 # amarillo
    elif user_accel < UMBRAL_CAMINA:
        estado = 'CAMINANDO'
        color = 0xFF8800 # naranja
    else:
        estado = 'INTENSO!'
        color = 0xFF0000 # rojo

    # 4. Actualizar pantalla
    label_acc.setText('Acc: {:.2f} {:.2f} {:.2f}'.format(ax, ay, az))
    label_mag.setText('|a|: {:.3f} g'.format(mag))
    label_user.setText('usr: {:.3f} g'.format(user_accel))
    label_estado.setText(estado)
    label_estado.setColor(color, 0x222222)

    if BtnA.wasHold():
        bias_gx, bias_gy, bias_gz = calibrar_giroscopio(200)

if __name__ == '__main__':
    try:
        setup()
        while True:
            loop()
    except (Exception, KeyboardInterrupt) as e:
        try:
            from utility import print_error_msg
            print_error_msg(e)
        except ImportError:
            print('Error:', e)
```

3.4 Experimentos de verificación

Experimento A — Tabla de estados

Acción realizada	user_accel observado	Estado detectado	¿Correcto?
M5 plano en mesa	_____	_____	
Agitar suavemente	_____	_____	
Caminar con M5	_____	_____	
Golpear la mesa	_____	_____	

Girar 180° lento	_____	_____	
------------------	-------	-------	--

Experimento B — Invariancia a la orientación

Con el M5 quieto, cambiad su orientación (boca abajo, de lado, inclinado). La magnitud $|a|$ debería mantenerse siempre ≈ 1.0 g y el estado debería seguir siendo QUIETO. Anotad las desviaciones:

Orientación	$ a $ medido	Desviación de 1g	Estado detectado
Plano, pantalla arriba	_____	_____	
Plano, pantalla abajo	_____	_____	
Vertical, pantalla al frente	_____	_____	
Inclinado 45°	_____	_____	

 Ajuste de umbrales
Si el estado 'QUIETO' se activa con vibraciones de la mesa o pequeñas vibraciones del suelo, aumentad UMBRAL_QUIETO de 0.05 a 0.08 o 0.10.
Si 'CAMINANDO' no se detecta cuando camináis, bajad UMBRAL_CAMINA de 0.35 a 0.25.
Los umbrales óptimos dependen de dónde lleva el usuario el dispositivo (mano, bolsillo, muñeca).

**BLOQUE
4****Detector de pasos por detección de picos** **50 min**

4.1 Objetivo

Implementar desde cero un **detector de pasos basado en detección de picos** sobre la magnitud vectorial. El detector final debe contar pasos con precisión ± 1 en 10 pasos medidos manualmente, y calcular la cadencia en pasos por minuto.

4.2 Fundamento: el patrón de un paso

Al caminar, el ciclo de marcha produce un patrón característico en la magnitud vectorial:

- Fase de vuelo (pie en el aire): magnitud ligeramente $< 1g$
- Impacto del talón: pico de magnitud entre $1.2g$ y $1.8g$
- Apoyo del pie: magnitud $\approx 1g$ mientras el peso se transfiere
- Impulso / despegue: segundo pico más pequeño $\approx 1.1-1.3g$

El detector busca el descenso tras un pico (flanco bajante). Condiciones para validar un paso:

1. La magnitud en el pico debe superar un UMBRAL (normalmente $1.10-1.25 g$)
2. Deben haber pasado más de 250 ms desde el último paso detectado (anti-rebote)
3. No deben haber pasado más de 2000 ms (si pasa más tiempo, el movimiento no es marcha)

4.3 Algoritmo de detección explicado paso a paso

El algoritmo mantiene una máquina de estados con 3 variables principales:

Variable	Tipo	Significado
subiendo	bool	True si la señal lleva una tendencia ascendente
pico	float	Valor máximo alcanzado durante la subida actual
valle	float	Valor mínimo alcanzado durante la bajada actual

Transiciones de la máquina de estados:

- Si $mag > mag_anterior \rightarrow subiendo = True$, actualizar pico
- Si $subiendo=True$ y $mag < mag_anterior \rightarrow$ hemos superado el pico $\rightarrow$ evaluar si es un paso
- Si no subiendo $\rightarrow$ actualizar valle

4.4 Código completo del bloque 4

Cread el fichero `imu_pasos.py`:

```
PYTHON
import os, sys, io
import M5
from M5 import *
import time
import math

# =====
# PARÁMETROS DEL DETECTOR - ajustar empíricamente
# =====
UMBRAL = 1.12 # Magnitud mínima del pico para contar paso (g)
MIN_PASO_MS = 250 # Tiempo mínimo entre pasos (ms) - anti-rebote
MAX_PASO_MS = 2000 # Tiempo máximo entre pasos (ms) - marcha válida
N_MUESTRAS_CAL = 200 # Muestras para calibración del giroscopio
```

```

# =====
# ESTADO GLOBAL DEL DETECTOR
# =====
pasos          = 0
mag_anterior   = 0.0
subiendo       = False
pico           = 0.0
valle          = 0.0
ultimo_paso_ms = 0

# Para cadencia (pasos/min)
tiempos_pasos = []      # Lista de timestamps de los últimos 10 pasos
cadencia       = 0.0

# Para calibración
bias_gx = bias_gy = bias_gz = 0.0

# Widgets
title0 = lbl_pasos = lbl_cadencia = lbl_mag = lbl_estado = lbl_status = None

# =====
# RUTINA DE CALIBRACIÓN
# =====
def calibrar(n=N_MUESTRAS_CAL):
    lbl_status.setText('Calibrando...')
    lbl_status.setColor(0xFF8800, 0x222222)
    sx = sy = sz = 0.0
    for _ in range(n):
        g = Imu.getGyro()
        sx += g[0]; sy += g[1]; sz += g[2]
        time.sleep_ms(10)
    lbl_status.setText('Listo')
    lbl_status.setColor(0x888888, 0x222222)
    return sx/n, sy/n, sz/n

# =====
# FUNCIÓN CORE: DETECCIÓN DE PASO
# Devuelve True si se detectó un paso en esta muestra
# =====
def detectar_paso(mag, ahora_ms):
    global pasos, mag_anterior, subiendo, pico, valle
    global ultimo_paso_ms, tiempos_pasos, cadencia

    paso_detectado = False

    # — Máquina de estados —————
    if mag > mag_anterior:
        # Señal subiendo: actualizar el pico
        subiendo = True
        if mag > pico:
            pico = mag

    elif subiendo and mag < mag_anterior:
        # Hemos superado el máximo local (flanco bajante)
        subiendo = False

    # — Condiciones de validación del paso —————
    tiempo_desde_ultimo = time.ticks_diff(ahora_ms, ultimo_paso_ms)

    condicion_amplitud = (pico > UMBRAL)
    condicion_min_time = (tiempo_desde_ultimo > MIN_PASO_MS)
    condicion_max_time = (tiempo_desde_ultimo < MAX_PASO_MS or ultimo_paso_ms == 0)

    if condicion_amplitud and condicion_min_time and condicion_max_time:
        # ¡PASO VÁLIDO!
        pasos += 1
        ultimo_paso_ms = ahora_ms
        paso_detectado = True

        # Actualizar lista de tiempos para calcular cadencia
        tiempos_pasos.append(ahora_ms)
        if len(tiempos_pasos) > 10:
            tiempos_pasos.pop(0)      # Mantener solo los últimos 10

    # Cadencia = (n_pasos - 1) / tiempo_total en minutos

```

```

        if len(tiempos_pasos) >= 2:
            dt_ms = time.ticks_diff(tiempos_pasos[-1], tiempos_pasos[0])
            if dt_ms > 0:
                cadencia = (len(tiempos_pasos) - 1) / (dt_ms / 60000.0)

        # Resetear pico y valle para el próximo ciclo
        pico = mag
        valle = mag

    else:
        # Señal bajando: actualizar el valle
        if mag < valle:
            valle = mag

    mag_anterior = mag
    return paso_detectado

# =====
# SETUP
# =====
screen_count = 0

def setup():
    global title0, lbl_pasos, lbl_cadencia, lbl_mag, lbl_estado, lbl_status
    global bias_gx, bias_gy, bias_gz
    global pico, valle, mag_anterior

    M5.begin()
    Widgets.fillScreen(0x222222)

    title0 = Widgets.Title('Contador Pasos', 3, 0xFFFFFF, 0x0000FF, Widgets.FONTS.DejaVu18)
    lbl_pasos = Widgets.Label('Pasos: 0', 5, 35, 1.0, 0x00FF00, 0x222222, Widgets.FONTS.DejaVu24)
    lbl_cadencia = Widgets.Label('Cad: -- p/min', 5, 70, 1.0, 0xFFFF00, 0x222222, Widgets.FONTS.DejaVu12)
    lbl_mag = Widgets.Label('|a|: --', 5, 90, 1.0, 0xFF8800, 0x222222, Widgets.FONTS.DejaVu12)
    lbl_estado = Widgets.Label('', 5, 110, 1.0, 0x00FFFF, 0x222222, Widgets.FONTS.DejaVu18)
    lbl_status = Widgets.Label('Inicio...', 5, 145, 1.0, 0x888888, 0x222222, Widgets.FONTS.DejaVu12)

    bias_gx, bias_gy, bias_gz = calibrar()

    # Inicializar variables del detector
    acc = Imu.getAccel()
    mag_anterior = math.sqrt(acc[0]**2 + acc[1]**2 + acc[2]**2)
    pico = mag_anterior
    valle = mag_anterior

# =====
# LOOP
# =====
DT_MS = 20 # 50 Hz de muestreo

def loop():
    global screen_count, pasos, cadencia

    M5.update()

    # — Leer IMU —————
    acc = Imu.getAccel()
    ax, ay, az = acc[0], acc[1], acc[2]
    mag = math.sqrt(ax*ax + ay*ay + az*az)
    ahora = time.ticks_ms()

    # — Detectar paso —————
    paso = detectar_paso(mag, ahora)

    # — Actualizar pantalla (cada 5 ciclos = 10 Hz) —————
    screen_count += 1
    if paso or screen_count >= 5:
        screen_count = 0
        lbl_pasos.setText('Pasos: {}'.format(pasos))
        lbl_mag.setText('|a|: {:.3f} pico:{:.3f}'.format(mag, pico))

        if cadencia > 0:
            lbl_cadencia.setText('Cad: {:.0f} p/min'.format(cadencia))

        if paso:
            lbl_estado.setText('>>> PASO! <<<')
```

```

        lbl_estado.setColor(0x00FFFF, 0x222222)
    else:
        lbl_estado.setText('')

# — BtnA: resetear contador —————
if BtnA.wasPressed():
    pasos = 0
    cadencia = 0.0
    tiempos_pasos.clear()
    lbl_pasos.setText('Pasos: 0')
    lbl_cadencia.setText('Cad: -- p/min')

# — BtnB: recalibrar giroscopio —————
if BtnB.wasPressed():
    bias_gx, bias_gy, bias_gz = calibrar()

time.sleep_ms(DT_MS)

# =====
# MAIN
# =====
if __name__ == '__main__':
    try:
        setup()
        while True:
            loop()
    except (Exception, KeyboardInterrupt) as e:
        try:
            from utility import print_error_msg
            print_error_msg(e)
        except ImportError:
            print('Error:', e)

```

4.5 Procedimiento de calibración del umbral

El parámetro más crítico es `UMBRAL`. Seguir este procedimiento para ajustarlo:

4. Ejecutar el código con `UMBRAL = 1.08` (valor inicial conservador)
5. Caminar exactamente 10 pasos y observar el contador
6. Si el contador da menos de 10: bajar `UMBRAL` en 0.02 y repetir
7. Si el contador da más de 10 (falsos positivos): subir `UMBRAL` en 0.02 y repetir
8. Repetir hasta conseguir 10/10 de forma consistente

UMBRAL probado	Pasos contados (de 10)	Falsos positivos	Pasos perdidos	¿Aceptable?
1.08	_____	_____	_____	
1.10	_____	_____	_____	
1.12	_____	_____	_____	
1.15	_____	_____	_____	
Valor final: ____	_____	_____	_____	

4.6 Experimentos de evaluación

Experimento A — Precisión en condiciones normales

Prueba	Pasos reales	Pasos contados	Error	Contexto
1	10	_____	_____	Caminando en línea recta

2	20	_____	_____	Caminando en línea recta
3	10	_____	_____	Subiendo escaleras
4	10	_____	_____	M5 en el bolsillo
5	10	_____	_____	Paso lento (1 paso/2s)

Experimento B — Prueba de cadencia

Caminad durante 30 segundos a ritmo constante. Anotad la cadencia que muestra el detector y comparadla con vuestra estimación manual (contar pasos durante 10 s × 6):

Métrica	Valor detector	Valor manual	Diferencia
Cadencia (p/min)	_____	_____	_____
Total pasos en 30s	_____	_____	_____

i Controles del detector en tiempo real

BtnA (pulsación corta): Resetea el contador de pasos a 0

BtnB (pulsación corta): Recalibra el giroscopio (dispositivo en reposo)

Probad el reset a mitad de una prueba y verificad que el conteo reinicia correctamente.

4.7 Comparativa con contador_pasos.py

Ahora cargad el fichero `contador_pasos.py` del proyecto y repetid exactamente la prueba del Experimento A. Comparad los resultados:

Aspecto	Vuestro detector (imu_pasos.py)	contador_pasos.py
Algoritmo de filtrado	Ninguno (magnitud directa)	BandPass (HighPass + LowPass)
Rango de frecuencias	Todas (0 – Nyquist)	0.5 Hz – 5 Hz únicamente
Sensibilidad a vibración mesa	_____	_____
Precisión en 10 pasos	_____	_____
Falsos positivos al sentarse	_____	_____
Código (líneas aprox.)	_____	≈ 130 líneas

El filtro BandPass de `contador_pasos.py` acepta sólo frecuencias entre 0.5 Hz y 5 Hz, que corresponden exactamente al rango de la marcha humana (0.5–3 pasos/s). Esto elimina:

- Vibraciones rápidas del motor o la mesa (> 5 Hz) → el filtro pasa-bajo las elimina
- Variaciones lentas de postura o gravedad (< 0.5 Hz) → el filtro pasa-alto las elimina

☒ ¿Por qué usamos BandPass en lugar de umbral simple?

Con umbral simple, cualquier vibración brusca que supere el UMBRAL cuenta como paso.

Con BandPass, sólo las oscilaciones en la frecuencia de marcha (0.5–5 Hz) llegan al detector.

Resultado: menos falsos positivos al sentarse, teclear, o vibrar sobre superficies irregulares.

El precio es mayor complejidad de código y un pequeño retardo de procesamiento (≈ 2 muestras).

Cierre: Debate y preguntas de reflexión

Dedicad los últimos 10 minutos a responder estas preguntas en grupo:

Pregunta 1 — Integración e inercia

El Tema 3 demuestra que un bias de 0.01g integrado durante 30 segundos produce un error de posición de **45 metros**. Habéis observado en el Bloque 2 el bias real de vuestro giroscopio. Calculad: si el giroscopio tiene un bias residual de 0.5°/s después de la calibración, ¿cuántos grados de error acumula tras 60 segundos de integración?

$$\text{Error_giro} = \text{bias} \times \text{tiempo} = \underline{\hspace{2cm}} \text{ } ^\circ/\text{s} \times 60 \text{ s} = \underline{\hspace{2cm}} \text{ } ^\circ$$

Respuesta: _____

Pregunta 2 — PDR vs integración doble

Explicad con vuestras propias palabras por qué PDR (Pedestrian Dead Reckoning) detectando pasos es más viable que integrar la aceleración directamente para obtener posición:

Respuesta: _____

Pregunta 3 — Fusión sensorial

Si se dispusiera de señal WiFi o Bluetooth para corregir la deriva, ¿cómo combinaríais la detección de pasos con las mediciones de RSSI para localización? ¿Qué aporta el acelerómetro que el WiFi no puede dar? Recordad el concepto de **ZUPT (Zero Velocity Update)** del Tema 3.

Respuesta: _____

Pregunta 4 — Mejoras al detector

Listáis al menos 3 mejoras que aplicaríais al detector imu_pasos.py para acercarlo a la calidad de contador_pasos.py:

9. _____

10. _____

11. _____

Apéndice: Resumen de ficheros y controles

Fichero	Bloque	Descripción
IMU.py	1 (base)	Visualización básica Acc y Gyro sin procesamiento

imu_calibrado.py	2	Añade calibración estática del giroscopio
imu_magnitud.py	3	Añade magnitud vectorial e indicador de actividad
imu_pasos.py	4	Detector de pasos por detección de picos
contador_pasos.py	Referencia	Detector con filtro BandPass (referencia del proyecto)

Botón / Acción	imu_calibrado.py	imu_magnitud.py	imu_pasos.py
BtnA (corto)	—	—	Resetear contador
BtnA (largo)	Recalibrar	Recalibrar	—
BtnB (corto)	—	—	Recalibrar giroscopio

i Referencia: Secciones del manual UIFlow 2.0

Sección 2 — M5 Sistema Base: M5.begin(), M5.update()

Sección 3 — Widgets: Label.setText(), Label.setColor(), Widgets.fillScreen()

Sección 4 — Botones: BtnA.wasPressed(), BtnA.wasHold(), BtnB.wasPressed()

Sección 6 — IMU: Imu.getAccel(), Imu.getGyro() — valores en g y dps

Sección 13 — time: time.ticks_ms(), time.ticks_diff(), time.sleep_ms()

Sección 21 — Math: math.sqrt(), math.atan2(), math.degrees()